

Advanced ANN Project

Autoencoder 기반 이상 탐지 모델의 성능 향상을 위한
하이퍼파라미터 최적화 연구

전자정보공학부 IT융합전공

20230438 성도연

20232095 유지원

20232065 이다정

20212861 이재민

Contents

- 01 연구 배경 및 목적
- 02 Autoencoder기반 이상 탐지
- 03 Baseline 모델 설정 및 한계
- 04 하이퍼파라미터 최적화 실험
- 05 최종 모델

▷ 데이터 기반 시스템에서 이상 탐지의 중요성

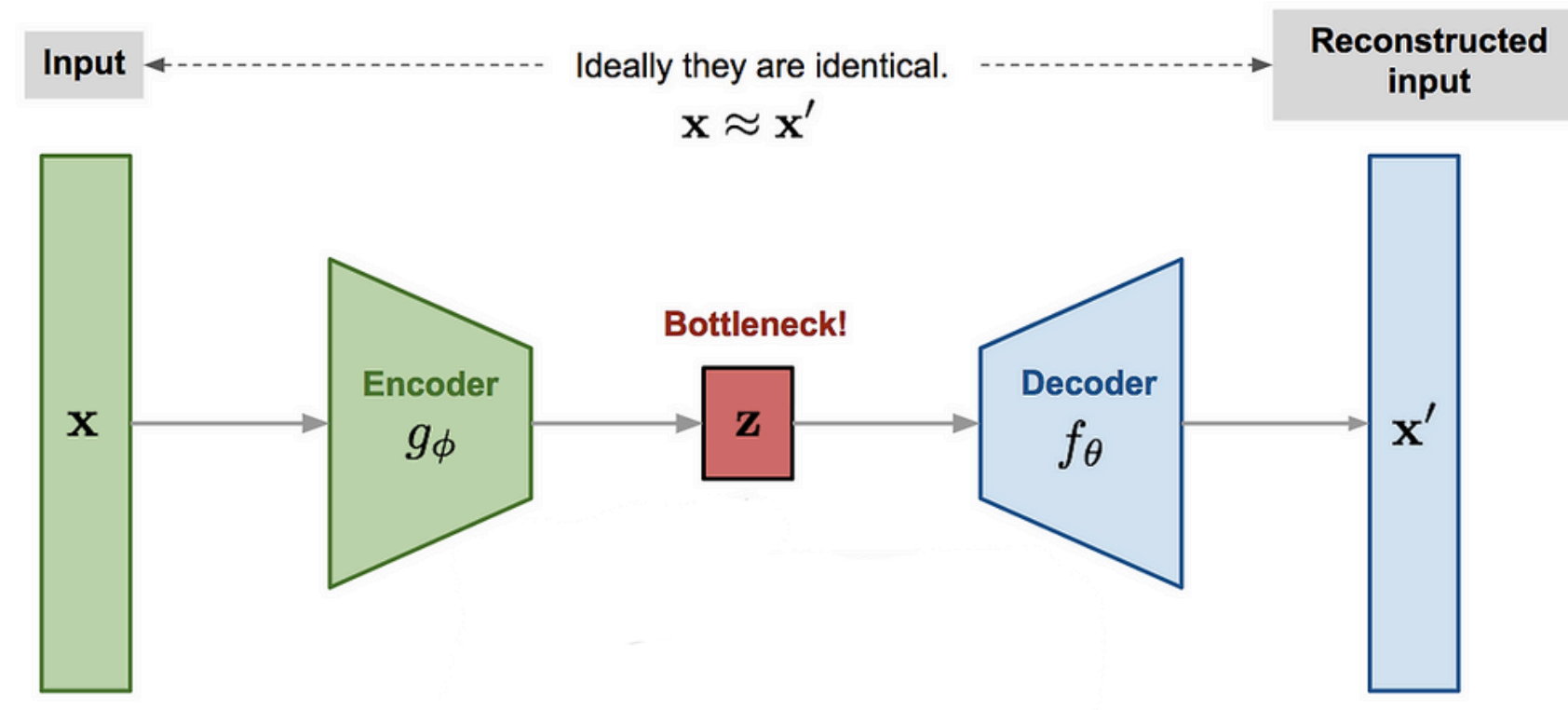
실제 환경에서는 이상 데이터가 매우 드물고, 레이블도 충분하지 않은 경우가 많음

Autoencoder는 정상 데이터를 복원한 뒤, 재구성 오차를 이용해 이상 여부를 판단하는 데 활용됨

▷ 목적

Autoencoder 기반 이상 탐지 모델의 하이퍼파라미터들을 단계적으로 최적화하여, 가장 안정적으로 수렴하고 높은 탐지 정확도를 보이는 최적의 모델 구성을 도출함

각 하이퍼파라미터가 모델의 수렴성과 오차분포에 어떤 영향을 미치는지 분석함



▷ Autoencoder

입력 x 를 잠재 공간으로 압축 후 복원하는($\hat{x}$) 비지도 신경망

▷ 이상 탐지에 적합한가?

정상 데이터 : 복원이 잘 됨 → 재구성 오차 작음

이상 데이터 : 학습되지 않음 → 재구성 오차 큼

$$L(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

재구성 오차를 기준으로 threshold를 설정하여 이상 여부를 판단

Layer/Node 구조

- Layer=깊이, Node=너비
- 모델의 표현력 결정
- 패턴 학습을 강화하지만
지나치게 크면 정상 데이터의 세부
잡음까지 학습하여 비정상 데이터의
재구성 오차가 줄어들 수 있음

Epoch

- 학습 반복 횟수
- 너무 크면 과적합 (이상까지 복원)
- 너무 작으면 학습 부족

Batch Size

- 한번 업데이트 시 사용하는 샘플 수
- 너무 크면 오차가 평탄해져 성능 하락
- 적당히 작으면 gradient noise 증가
→ 일반화 성능 향상

Learning Rate

- 학습 속도 · 파라미터 업데이트 크기
- 수렴 안정성 결정
- 크면 초기에 빠르게 수렴하지만 불안정
- 낮으면 안정적이나 수렴 속도 느림

Activation Function

- Layer의 비선형성,
Gradient 흐름의 안정성에 영향
- ReLU: 음수=0, 양수=통과
단순, 효율적 계산
gradient 소실 가능성
- GELU: 입력을 확률적으로 통과시킴
비선형적 학습
- SiLU: 입력 * sigmoid
→ 부드러운 활성화

Optimizer

- 가중치 업데이트
→ 수렴 속도 · 안정성 결정
- Adam: 모멘텀+적응적 학습률 사용
수렴이 빠르지만 진동 가능
- AdamW: weight decay를 분리해
일반화 성능 보완
- NAdam: Nesterov 모멘텀을 추가해
진동을 줄이고 더 안정적으로
수렴

Threshold

- 재구성 오차를 기준으로
정상/비정상을 구분하는 기준
- 각 모델의 오차 분포에서 자동 산출된
값을 사용
→ 하이퍼파라미터 조합이 오차
분포에 미치는 효과를 평가하기 위함

▷ Accuracy (정확도)

전체 샘플 중 올바르게 분류된 비율. 모델의 전반적인 성능

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

▷ Precision (정밀도)

모델이 '이상'이라 예측한 샘플 중 실제로 이상일 확률

$$Precision = \frac{TP}{TP + FP}$$

▷ Recall (재현율)

모델이 실제 이상 중 올바르게 탐지한 비율
FN(이상을 놓치는 경우)을 줄이는 데 중요함

$$Recall = \frac{TP}{TP + FN}$$

▷ F1-score

Precision과 Recall의 조화평균
두 지표 간 균형을 평가함

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

“ 실제 불량 탐지는 극대화하되, 과도한 오검출은 줄이자! ”

Layer/Node	Activation Function	Optimizer	Learning Rate	Batch Size	Epoch	Loss Function	Threshold
784 → 596 → 408 → 220 → 32	GELU	Adam	5e-3	64	400	MSE	자동 산출 (Val Reconstruction Error)

	Accuracy	Precision	Recall	F1-Score	Train loss
Val	0.8348	0.8818	0.7733	0.8240	0.0069
Test	0.7717	0.7678	0.7785	0.7731	

- ▷ 정상 데이터를 충분히 학습하지 못해 재구성 오차 분포가 넓게 나타나
정상·이상 구분이 명확하지 않음

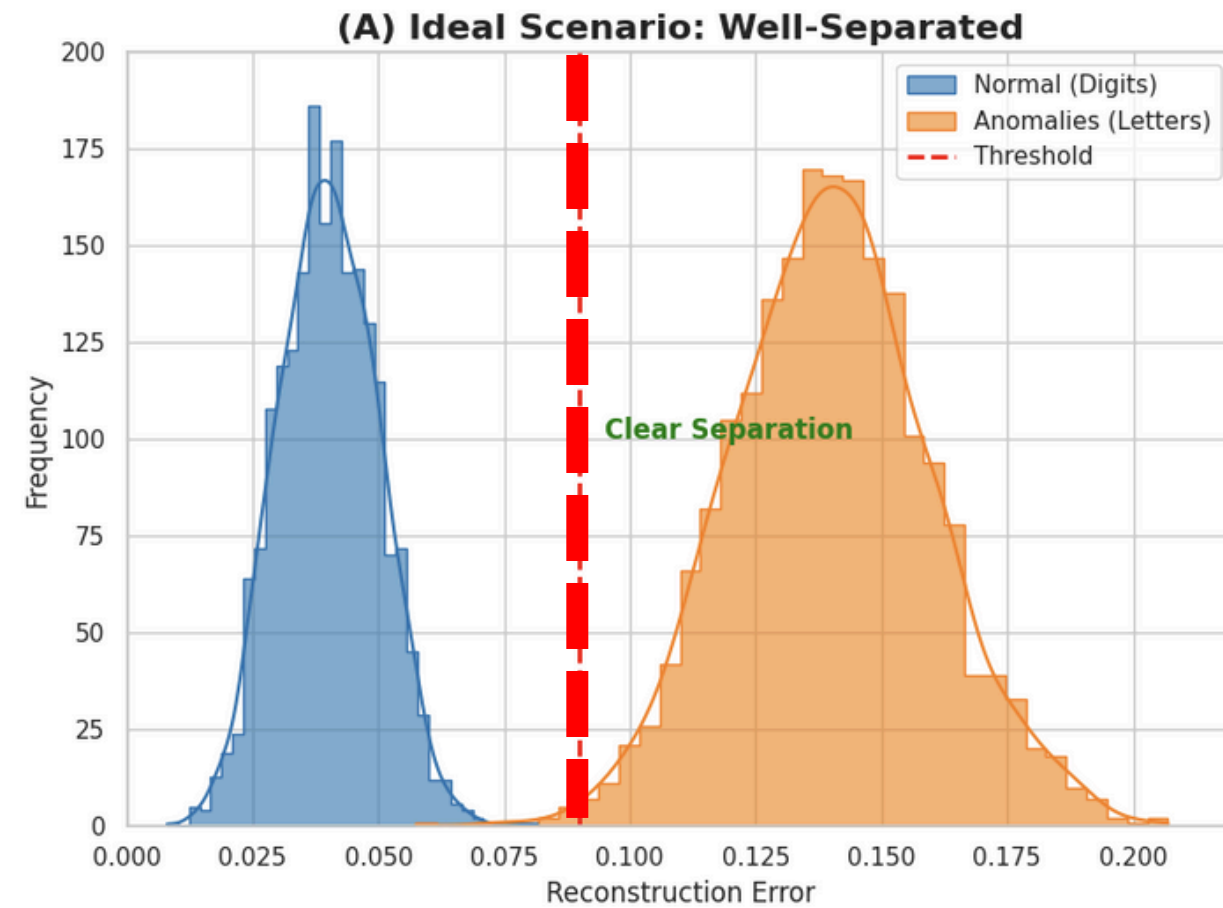
Test	Accuracy	Precision	Recall	F1-Score	Train loss
784-596-408-220-32 (기본)	0.7839	0.7793	0.7920	0.7856	0.0082
784-512-256-128-32	0.7790	0.7754	0.7855	0.7804	0.0090
784-512-256-128-64-32	0.7837	0.8007	0.7555	0.7775	0.0111
784-600-512-256-128-64-32	0.7994	0.8081	0.7853	0.7965	0.0187

▷ Layer와 Node 수 확장에 따라 Precision, F1-score, Accuracy ↑, Recall ↓

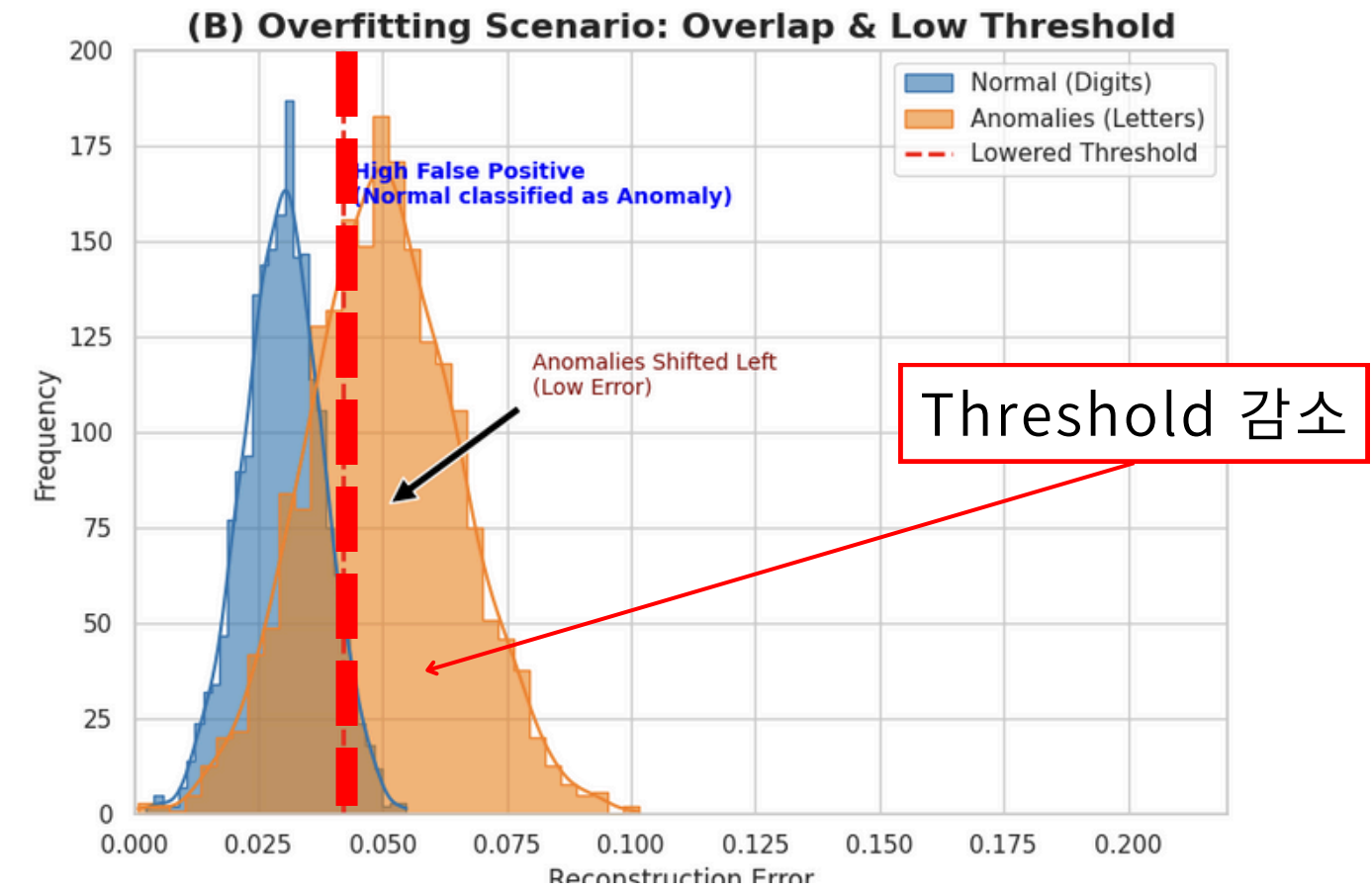
- 확장 전
 - recall만 증가
 - 층이 얇음 → Latent Space 분리 성능 저하 → 낮은 Representation Capacity
- 확장 후
 - recall 감소 및 precision 증가
 - Latent Space 분리가 선명해짐
 - 정상 데이터의 패턴을 더 정교하게 복원
 - 이상 데이터는 reconstruction error가 커지며 정상/이상 구분이 더욱 뚜렷해짐
- 결론 : Precision 증가와 Recall 감소가 상호 보완되어 F1-score와 Accuracy가 향상

- ▷ epoch 400의 confusion matrix
- ▷ Recall을 제외하고 Validation 성능에 비해 매우 낮아진 나머지 Test 지표

	Accuracy	Precision	Recall	F1-Score	Train loss
Val	0.9450	0.9777	0.9108	0.9431	0.0025
Test	0.6786	0.6220	0.9105	0.7391	



▷ 이상적인 경우



- ▷ 과적합 시 이상 데이터의 재구성 오차가 낮아짐
- ▷ 정상/이상 분류를 위해 Threshold가 과도하게 낮아짐
 - 모델이 너무 민감해져 FN 감소, TP & FP 증가

▷ epoch 60

	Accuracy	Precision	Recall	F1-Score	Train loss
Val	0.8613	0.9284	0.7829	0.8495	0.0082
Test	0.7839	0.7793	0.7920	0.7856	

하이퍼파라미터 최적화 실험- Batch size, Learning Rate

16

LR / Batch	Accuracy	Precision	Recall	F1-Score	Train loss
0.001 / 64	0.8083	0.8010	0.8203	0.8105	0.0110
0.001 / 128	0.8005	0.7892	0.8200	0.8043	0.0130
0.001 / 256	0.8030	0.8081	0.7947	0.8014	0.0133
0.005 / 64	0.7994	0.8081	0.7853	0.7965	0.0187
0.005 / 128	0.5116	0.5060	0.9752	0.6663	0.1592
0.005 / 256	0.6141	0.6291	0.5560	0.5903	0.0821

▷ 작은 lr(0.001)
: batch의 변화에 안정적

▷ 큰 lr(0.005)
: batch 커질 수록 gradient 불안정
→ 성능 붕괴

하이퍼파라미터 최적화 실험- Batch size, Learning Rate

17

LR / Batch	Accuracy	Precision	Recall	F1-Score	Train loss
0.001 / 64	0.8083	0.8010	0.8203	0.8105	0.0110
0.001 / 128	0.8005	0.7892	0.8200	0.8043	0.0130
0.001 / 256	0.8030	0.8081	0.7947	0.8014	0.0133
0.005 / 64	0.7994	0.8081	0.7853	0.7965	0.0187
0.005 / 128	0.5116	0.5060	0.9752	0.6663	0.1592
0.005 / 256	0.6141	0.6291	0.5560	0.5903	0.0821

▷ 작은 batch
: 다양한 noise가 섞여
local minima&과적합 방지

▷ 큰 batch
: gradient 변동이 작고 안정적.
noise가 적어 local minima에
수렴하기 쉬움
→ test 성능 ↓

큰 학습률: 업데이트 스텝 ↑

+

큰 배치 크기: gradient의 분산 감소

↓

- 1) 최소점 주변을 지나쳐 버리고 진동하거나 발산
- 2) 큰 배치로 무작위성 줄어들어 local minima로 수렴

- ▷ 다른 조합 대비 정상·이상 데이터 재구성 오차의 분리도가 우수
- ▷ 비교적 Recall값이 낮지만 전체 지표가 가장 균형적

Test	Accuracy	Precision	Recall	F1-Score	Train loss
Adam+SiLU	0.8083	0.8010	0.8203	0.8105	0.0110
AdamW+GELU	0.8033	0.7934	0.8200	0.8065	0.0129
NAdam+ReLU	0.8116	0.8133	0.8090	0.8111	0.0128

경계가 뚜렷하고 정규화된 EMNIST 데이터에서

ReLU 의 희소성(Sparsity) 유도 능력이 핵심 특징을 추출함

+

NAdam의 선제적인 모멘텀 보정

↓

정상·이상 오차 분포의 간극을 크게 만듦

“ 성능은 복잡한 활성화 함수보다,
데이터·모델·학습 조건의 조합이 좌우한다 ”

▷ L1

- 오차 크기와 무관하게 기울기가 1, -1로 동일
- 학습 후반부에 수렴하지 못하고 미세한 진동 발생

▷ HuberLoss

- 임계점 설정의 어려움 존재
- 큰 오차에 강한 패널티를 주는 MSE보다 식별력이 떨어짐

▷ BCELoss

- Sigmoid() 사용 → 입력을 0~1 범위의 확률로 변환해 오차 계산
 - 값을 과도하게 압축하여 비선형 왜곡 유발
-

Test	Accuracy	Precision	Recall	F1-Score	Train loss
MSE	0.8116	0.8133	0.8090	0.8111	0.0128
HuberLoss	0.8024	0.7890	0.8255	0.8068	0.0062
L1	0.8080	0.8128	0.8003	0.8065	0.0474
BCELoss	0.8039	0.7899	0.8280	0.8085	0.1482

▷ MSE Loss

- 큰 오차에 큰 패널티 → 모델이 빠르게 학습
- 작은 오차는 더 작게 → 안정적인 수렴과 미세 조정 가능
- Convex → local minima에 빠질 확률 없음

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Layer/Node	Activation Function	Optimizer	Learning Rate	Batch Size	Epoch	Loss Function	Threshold
784 → 600 → 512 → 256 → 128 → 64 → 32	ReLU	NAdam	1e-3	64	60	MSE	자동 산출 (ACC 우선)

	Accuracy	Precision	Recall	F1-Score	Train loss
Val	0.8708	0.9282	0.8038	0.8615	0.0128
Test	0.8116	0.8133	0.8090	0.8111	

→ 가장 안정적이고
높은 재현성

Thank you